

Institut National des Sciences Appliquées et de Technologie

INSAT

Année Universitaire 2025 – 2026

Conception d'un système multi-agent intelligent pour le traitement de demandes utilisateurs dans une plateforme e-commerce

Rapport de Projet de Fin d'Année

Réalisé par :

Kousay Jebir
Chedli Masmoudi
Ahmed Gargouri
Yasmine Zerai

Encadré par :

Mme Hajer Taktak

Table des matières

1 Cadre du projet **1**

1.1 Contexte et motivation 1
1.2 Problématique 2
1.3 Objectifs du projet 3

2 Étude des solutions existantes et limites **4**

2.1 Support client traditionnel 4
2.2 Chatbots à base de règles 5
2.3 Assistants conversationnels basés sur les LLMs 5
2.4 Approches RAG 6
2.5 Architectures multi-agents 7
2.6 Notre positionnement 8

3 Conception et architecture du système **10**

3.1 Analyse des besoins 10
3.1.1 Présentation du cas d'usage 10
3.1.2 Acteurs du système 11
3.1.3 Besoins fonctionnels 11
3.1.4 Besoins non fonctionnels 12
3.1.5 Diagramme de cas d'utilisation 12
3.2 Décisions de conception 13
3.3 Architecture globale 14
3.4 Indépendance vis-à-vis des sources de données 16

4 Données et environnement documentaire **18**

4.1 Données documentaires 18
4.1.1 Le problème des données documentaires 18
4.1.2 Pipeline de génération synthétique 19
4.1.3 Le document source — Manuel StyleSphere 20
4.1.4 Infrastructure RAG — de la politique à la décision 22
4.2 Données transactionnelles 24

4.2.1 Architecture de stockage 24
4.2.2 Données de test — conception orientée scénarios 24

5 L'orchestrateur 27
5.1 Architecture interne — les trois nœuds 28
5.2 Phase 1 — Détection des intentions 29
5.3 Phase 2 — Slot-filling 29
5.4 Phase 3 — Routage et exécution 30
5.4.1 Sentiment override 31
5.4.2 Le cycle pending_customer 32
5.4.3 Combine node 32
5.5 Contrats orchestrateur-agents 32
5.6 Tests et évaluation 34
5.6.1 Scénario 1 — Requête multi-intentions sur deux commandes distinctes 34
5.6.2 Scénario 2 — Client ne se souvient pas de son identifiant de commande 35
5.6.3 Scénario 3 — Acquittement simple 35

6 Les agents spécialisés 37
6.1 BaseAgent — la boucle partagée 37
6.1.1 L'invariant action-blocking 37
6.2 Complaint Agent 39
6.2.1 Conception 39
6.2.2 Tests 40
6.3 Order Agent 41
6.3.1 Conception 41
6.3.2 Tests 41
6.4 FAQ Agent 42
6.4.1 Conception 42
6.4.2 Tests 42
6.5 Sales Agent 43
6.5.1 Conception 43
6.5.2 Tests 43
6.6 Escalation Agent 44
6.6.1 Conception 44
6.6.2 Tests 44

7 Réalisation et perspectives 45
7.1 Stack technologique 45
7.2 Le système multi-agent 45
7.3 La couche de données pluggable 46

7.4 Les interfaces de démonstration 47
7.4.1 Dashboard administrateur 47
7.4.2 Widget chatbot client 48
7.5 Projection produit 49
7.5.1 Un système conçu pour être vendu 49
7.5.2 Cas d'usage réels envisagés 49
7.5.3 Roadmap vers un déploiement réel 50

Conclusion **51**

Table des figures

2.1 Arbre de décision d'un chatbot à base de règles 5
2.2 Pipeline Retrieval-Augmented Generation 6
2.3 Architecture multi-agents avec orchestrateur central 7
2.4 Positionnement de notre solution 8

3.1 Diagramme de cas d'utilisation du système 12
3.2 Architecture globale du système 14

4.1 Pipeline de génération synthétique de corpus documentaires 19
4.2 Pipeline d'indexation et de retrieval du service PFARAG 22
4.3 Distribution des données de test par rapport aux seuils de politique StyleSphere 25

5.1 Graphe d'exécution LangGraph de l'orchestrateur 28
5.2 Flux d'exécution du routing$_n$*ode* 30

6.1 Boucle agentique BaseAgent 38

7.1 Exemple d'éxecution de l'agent Order 46
7.2 Dashboard administrateur — vue Tickets 47
7.3 Dashboard administrateur — vue RAG 47
7.4 Widget chatbot — interface client de démonstration 48
7.5 Roadmap produit — trois horizons 50

Liste des tableaux

1.1 Évolution des enjeux du support client e-commerce 1
1.2 Objectifs du projet et indicateurs de réussite 3

2.1 Comparaison des approches traditionnelles de support client 4
2.2 Limites d'un agent LLM unique en contexte e-commerce 6
2.3 Avantages des architectures multi-agents 7

3.1 Acteurs du système et leurs rôles 11
3.2 Besoins fonctionnels du système 11
3.3 Besoins non fonctionnels et leur réalisation 12

4.1 Étapes de la pipeline de génération synthétique 20
4.2 Sections du manuel StyleSphere et agents concernés 21
4.3 Pipeline de prétraitement et d'indexation 22
4.4 Distribution des données de test et scénarios couverts 25

5.1 Les trois nœuds de l'orchestrateur 28
5.2 Champs requis par intention 30
5.3 Étapes du routing_node 31
5.4 Table de routage INTENT_TO_AGENT 31
5.5 Déroulement du cycle pending_customer 32
5.6 Déroulement — deux commandes distinctes 34
5.7 Déroulement — order_id inconnu du client 35
5.8 Déroulement — message d'acquittement 35

6.1 Classification des outils — lecture vs action 38
6.2 Matrice de décision du Complaint Agent 40
6.3 Scénarios de test — Complaint Agent 40
6.4 Scénarios de test — Order Agent 41
6.5 Scénarios de test — FAQ Agent 42
6.6 Scénarios de test — Sales Agent 43
6.7 Scénarios de test — Escalation Agent 44

7.1 Technologies utilisées dans le projet 45
7.2 Pages du dashboard et informations disponibles 48
7.3 Cas d'usage potentiels en production 49

Chapitre 1

Cadre du projet

Introduction

Ce chapitre présente le contexte dans lequel s'inscrit notre projet, la problématique qu'il cherche à résoudre, et les objectifs que nous nous sommes fixés. Il constitue le point de départ de notre démarche de conception d'un système multi-agent intelligent pour le support client e-commerce.

1.1 Contexte et motivation

Le commerce électronique connaît une croissance exponentielle. Les plateformes doivent aujourd'hui traiter des milliers de demandes clients simultanément, couvrant des domaines variés et souvent complexes.

TABLE 1.1 – Évolution des enjeux du support client e-commerce

Dimension	Avant	Aujourd'hui
Volume de demandes	Faible, gérable manuellement	Massif, pics saisonniers
Complexité	Questions simples	Multi-étapes, multi-intentions
Délai attendu	Heures / jours	Immédiat (24/7)
Coût humain	Acceptable	Difficilement scalable
Connaissance requise	Limitée	Politiques internes complexes

Face à cette réalité, les approches traditionnelles montrent leurs limites. Les **agents humains** offrent une flexibilité remarquable mais leur scalabilité est limitée par le coût et la disponibilité. Les **chatbots à base de règles** automatisent les cas simples mais échouent dès que la demande sort des scénarios prévus.

L'émergence des **Large Language Models (LLMs)** a ouvert de nouvelles perspectives, permettant de comprendre le langage naturel et de générer des réponses cohérentes. Mais un agent LLM unique présente ses propres limites dans un contexte e-commerce réel :

- Il ne connaît pas les politiques internes de l'entreprise
- Il ne peut pas gérer simultanément des intentions multiples
- Il manque de spécialisation pour des domaines comme la fraude ou les remboursements
- Il peut *halluciner* des informations incorrectes

C'est dans ce contexte que les architectures **multi-agents** combinées aux approches **RAG** apparaissent comme la solution adaptée.

1.2 Problématique

Problématique centrale

Comment concevoir un système de support client automatisé capable de traiter des demandes complexes et multi-intentions dans un environnement e-commerce, tout en respectant des politiques métier spécifiques et en maintenant un contexte cohérent sur l'ensemble de la conversation ?

Cette problématique centrale se décompose en trois défis techniques :

Défi 1

Spécialisation

Un système unique gérant toutes les tâches devient ingérable. Les responsabilités doivent être réparties entre agents spécialisés.

Défi 2

Connaissances documentaires

Les réponses doivent rester cohérentes avec les politiques internes, interrogeables dynamiquement sans réentraînement.

Défi 3

Contexte conversationnel

Une conversation peut contenir plusieurs intentions. Le système doit maintenir la cohérence et savoir escalader vers un humain.

1.3 Objectifs du projet

L'objectif principal est de concevoir et implémenter un système multi-agents intelligent pour automatiser le support client d'une plateforme e-commerce.

TABLE 1.2 – Objectifs du projet et indicateurs de réussite

Objectif	Indicateur de réussite
Spécialisation des agents	Chaque agent traite uniquement son domaine et retourne un `AgentResult` structuré
Intégration RAG	Les agents consultent la base documentaire avant toute décision de politique
Multi-intentions	Le système détecte et traite plusieurs intentions dans un seul message client
Modularité	Changer de base de données ne nécessite aucune modification des agents
Traçabilité	Chaque interaction génère un ticket d'audit
Escalade intelligente	Les cas hors-portée sont transmis à un humain avec contexte complet

Notre solution s'articule autour de quatre piliers :

1. **Architecture multi-agents** — un orchestrateur central coordonne cinq agents spécialisés
2. **Pipeline RAG** — les politiques métier sont indexées dans une base vectorielle interrogée dynamiquement avant chaque décision
3. **Ports & Adapters** — les agents sont totalement découplés de la base de données
4. **Données synthétiques** — un environnement documentaire et transactionnel réaliste pour évaluer le système dans des conditions proches du réel

Conclusion

Ce chapitre a posé les bases de notre projet en présentant le contexte e-commerce, les limites des solutions existantes, et les trois défis techniques que nous cherchons à résoudre. Le chapitre suivant passe en revue les solutions existantes dans la littérature et positionne notre approche par rapport à l'état de l'art.

Chapitre 2

Étude des solutions existantes et limites

Introduction

Avant de présenter notre solution, il est essentiel de comprendre pourquoi les approches existantes ne suffisent pas. Ce chapitre passe en revue les différentes générations de systèmes de support client, de l'opérateur humain aux architectures multi-agents, en identifiant pour chacune les limites qui motivent notre approche.

2.1 Support client traditionnel

Le support client a évolué en plusieurs générations, chacune cherchant à pallier les limites de la précédente.

TABLE 2.1 – Comparaison des approches traditionnelles de support client

Approche	Avantages	Limites
Agent humain	Flexibilité, empathie, gestion de l'ambiguïté	Coût élevé, non scalable, incohérence entre agents
Centre d'appel	Organisation, suivi, historique centralisé	Dépendance humaine totale, temps de traitement longs
Système de tickets	Traçabilité, catégorisation	Passif — ne traite pas, ne répond pas automatiquement

Limite fondamentale Ces approches ne passent pas à l'échelle. Un pic de demandes (soldes, incidents logistiques) nécessite de recruter temporairement, ce qui est coûteux et difficile à maintenir en qualité.

2.2 Chatbots à base de règles

Les chatbots classiques ont été la première réponse à ce problème d'échelle. Ils reposent sur des arbres de décision (Figure 2.1) et des expressions régulières .

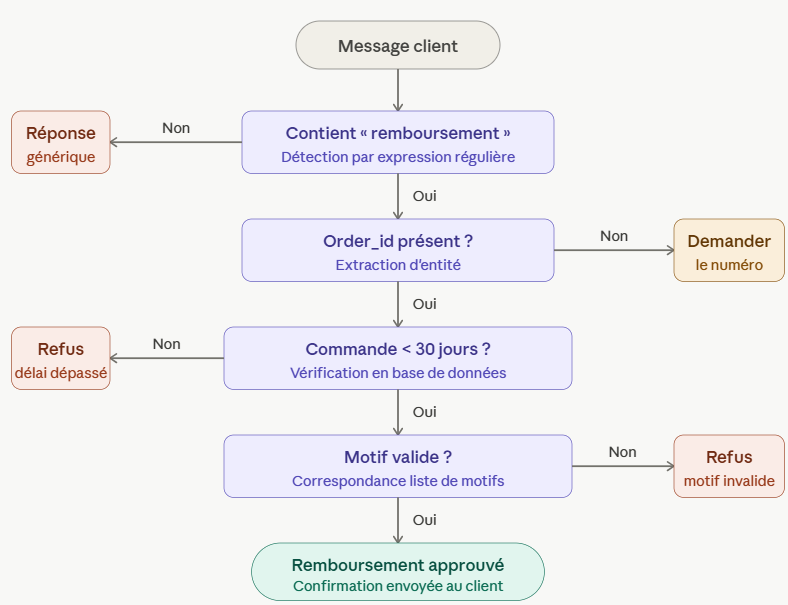


FIGURE 2.1 – Arbre de décision d'un chatbot à base de règles

+ Rapide à déployer sur des scénarios simples et répétitifs
+ Comportement entièrement prévisible et contrôlable
– Échec immédiat sur toute formulation non prévue
– Maintenance coûteuse — chaque nouvelle règle nécessite une intervention manuelle
– Aucune capacité de raisonnement documentaire

2.3 Assistants conversationnels basés sur les LLMs

L'émergence des **Large Language Models** (GPT-4, Claude, Llama) [1, 2] a radicalement transformé les capacités des systèmes conversationnels. Ces modèles comprennent le langage naturel, gèrent des conversations longues et produisent des réponses fluides.

TABLE 2.2 – Limites d'un agent LLM unique en contexte e-commerce

Problème	Conséquence concrète
Hallucinations	L'agent peut inventer une politique de remboursement inexistante
Absence de connaissances métier	Impossible de respecter des règles internes sans les fournir explicitement à chaque appel
Agent unique surchargé	Un seul prompt doit gérer commandes, fraude, retours et ventes — la qualité se dégrade
Pas de mémoire native	Sans mécanisme externe, le contexte est perdu entre les sessions
Cohérence non garantie	Deux questions identiques peuvent produire des réponses différentes

2.4 Approches RAG

Le **Retrieval-Augmented Generation (RAG)** [3] résout le problème de la connaissance métier en combinant un LLM avec une base documentaire interrogeable dynamiquement.

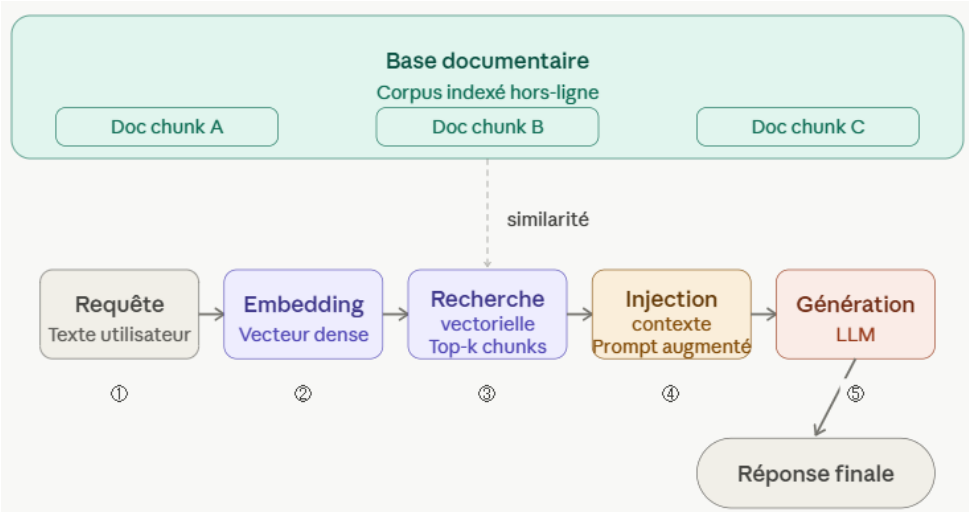


FIGURE 2.2 – Pipeline Retrieval-Augmented Generation

Le RAG apporte quatre bénéfices clés pour notre contexte (Figure 2.2) :

1. **Cohérence avec les politiques** — les réponses sont ancrées dans les documents réels de l'entreprise
2. **Mise à jour sans réentraînement** — changer une politique revient à mettre à jour un document, pas un modèle

3. **Traçabilité** — chaque réponse peut être reliée à sa source documentaire
4. **Réduction des hallucinations** — le modèle répond à partir de chunks réels, pas d'inférences

Limite du RAG seul Le RAG enrichit un agent mais ne résout pas le problème de spécialisation. Un agent unique avec RAG reste surchargé dès que les domaines métier se multiplient.

2.5 Architectures multi-agents

Les architectures multi-agents [4, 5] répartissent les responsabilités entre plusieurs agents spécialisés coordonnés par un orchestrateur. La Figure 2.3 illustre cette organisation.

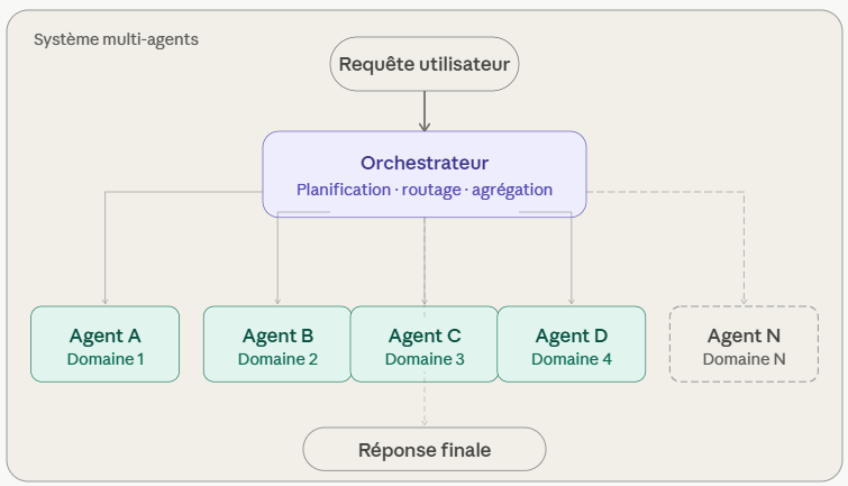


FIGURE 2.3 – Architecture multi-agents avec orchestrateur central

TABLE 2.3 – Avantages des architectures multi-agents

Propriété	Impact
Spécialisation	Chaque agent maîtrise un domaine précis — prompt plus court, raisonnement plus fiable
Modularité	Ajouter un nouvel agent ne modifie pas les agents existants
Maintenabilité	Un bug dans l'agent commandes n'affecte pas l'agent réclamations
Scalabilité	Les agents peuvent s'exécuter en parallèle
Traçabilité	Chaque agent produit un résultat structuré et auditable

2.6 Notre positionnement

Notre solution se situe à l'intersection de ces trois approches, comme illustré Figure 2.4 :

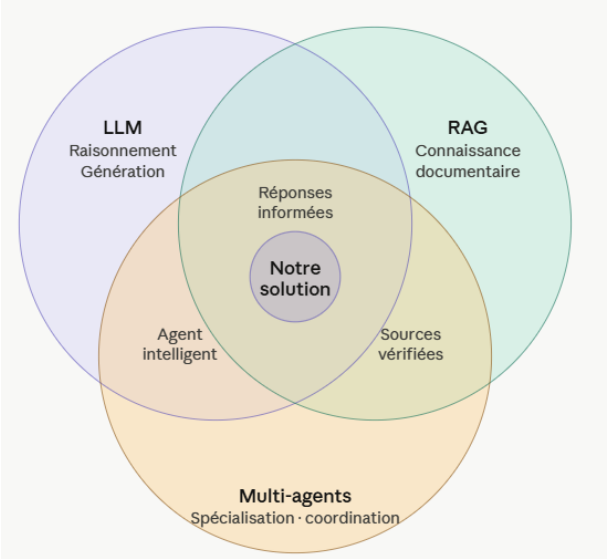


FIGURE 2.4 – Positionnement de notre solution

Critère	Humain	Règles	LLM seul	RAG seul	Notre solution
Scalabilité	✗	✓	✓	✓	✓
Connaissance métier	✓	✗	✗	✓	✓
Spécialisation	✓	✗	✗	✗	✓
Multi-intentions	✓	✗	~	✗	✓
Traçabilité	~	~	✗	~	✓
Escalade intelligente	✓	✗	✗	✗	✓

Notre approche

Un orchestrateur LangGraph coordonne cinq agents spécialisés, chacun enrichi par un pipeline RAG et connecté à la base de données via une architecture permettant de changer de politique sans toucher la logique agentique.

Conclusion

Ce chapitre a montré que chaque génération de solutions résout certains problèmes tout en en introduisant de nouveaux. Notre solution combine les trois approches les plus avancées — LLMs, RAG et multi-agents — pour adresser simultanément la scalabilité,

la spécialisation et la cohérence avec les politiques métier. Le chapitre suivant détaille la conception et l'architecture technique de ce système.

Chapitre 3

Conception et architecture du système

Introduction

Ce chapitre présente la conception complète du système, depuis l'analyse des besoins jusqu'aux choix d'architecture techniques. Nous détaillons les décisions de conception qui ont guidé le développement et les patterns architecturaux adoptés

3.1 Analyse des besoins

3.1.1 Présentation du cas d'usage

Notre système cible une plateforme e-commerce devant gérer automatiquement un large volume de demandes clients couvrant plusieurs domaines métier simultanément.

Cas d'usage principal

Un client envoie un message en langage naturel via un chatbot. Le système doit :

- Comprendre sa demande
- Consulter les politiques internes
- Interagir avec la base de données
- Retourner une réponse cohérente

Le tout sans intervention humaine, sauf cas exceptionnel.

3.1.2 Acteurs du système

TABLE 3.1 – Acteurs du système et leurs rôles

Acteur	Rôle	Interactions
Client	Utilisateur principal souhaitant une assistance	Envoie des messages, reçoit des réponses
Support humain	Opérateur prenant en charge les cas escaladés	Reçoit un résumé structuré, traite manuellement
Administrateur	Supervise les performances du système	Consulte le dashboard, gère les tickets ouverts

3.1.3 Besoins fonctionnels

TABLE 3.2 – Besoins fonctionnels du système

#	Besoin	Composant responsable
BF1	Analyser des demandes en langage naturel	Orchestrateur
BF2	Détecter et gérer des intentions multiples	Orchestrateur
BF3	Extraire les entités nécessaires (order_id, etc.)	Orchestrateur
BF4	Router vers l'agent spécialisé approprié	Orchestrateur
BF5	Interroger la base documentaire (politiques)	Agents + RAG
BF6	Lire et écrire dans la base de données	Agents + Adapters
BF7	Détecter les situations nécessitant une escalade	Agents + Sentiment
BF8	Conserver un historique des conversations	MongoDB + Tickets
BF9	Gérer le slot-filling (champs manquants)	Orchestrateur

3.1.4 Besoins non fonctionnels

TABLE 3.3 – Besoins non fonctionnels et leur réalisation

Propriété	Description	Solution adoptée
Modularité	Ajout d'un agent sans modifier les autres	Héritage BaseAgent
Maintenabilité	Composants clairement séparés	Ports & Adapters
Robustesse	Gestion des requêtes ambiguës	Slot-filling + escalade auto
Traçabilité	Audit de toutes les actions	create_ticket obligatoire
Cohérence	Réponses alignées avec les politiques	Pipeline RAG
Sécurité	Actions destructives contrôlées	Action-blocking invariant

3.1.5 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (Figure 3.1) représente l'ensemble des interactions entre les acteurs du système et les fonctionnalités qu'il expose. On y distingue trois acteurs principaux :le **client**, le **support humain** et l'**administrateur** .

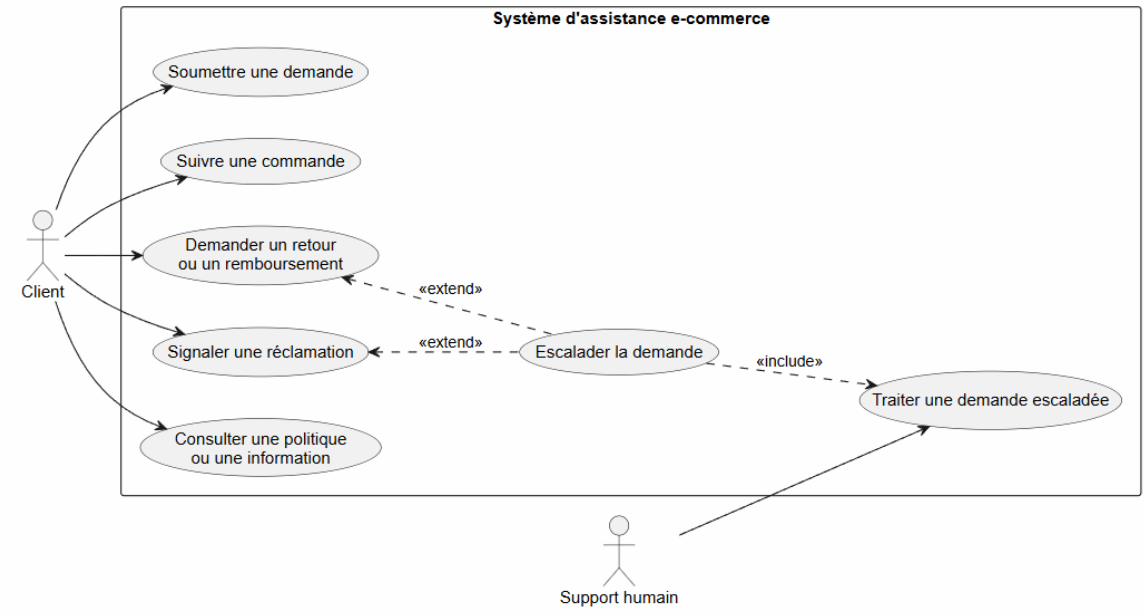


FIGURE 3.1 – Diagramme de cas d'utilisation du système

3.2 Décisions de conception

Avant de présenter l'architecture, nous exposons les quatre choix fondamentaux qui ont guidé notre conception. Pour chaque décision, nous présentons l'alternative écartée et la raison concrète qui a motivé notre choix.

01 Multi-agents spécialisés

Alternative écartée : agent LLM unique

Un agent unique chargé de gérer commandes, remboursements, fraude et ventes produit des prompts trop longs et des raisonnements peu fiables. Lors de nos tests, un agent unique confondait systématiquement la logique décisionnelle des remboursements avec celle des recommandations produits. La spécialisation résout ce problème structurellement.

02 RAG plutôt que fine-tuning

Alternative écartée : fine-tuning du modèle

Le fine-tuning nécessite des données d'entraînement et un réentraînement complet à chaque mise à jour de politique. Le RAG permet de mettre à jour les connaissances en modifiant uniquement les documents sans toucher au modèle, sans coût de réentraînement, avec une traçabilité complète des sources utilisées pour chaque décision.

03 Ports & Adapters

Alternative écartée : accès direct à la base

Les outils ne doivent pas connaître la source des données. Cette abstraction permet de basculer entre la base mock (développement, zéro infrastructure) et PostgreSQL (production) en changeant une seule variable d'environnement sans modifier une seule ligne de logique agentique. Le système est ainsi portable vers n'importe quel autre métier e-commerce.

04 LangGraph pour l'orchestration

Alternative écartée : boucle manuelle

LangGraph [6] fournit nativement le checkpointing, la persistance d'état entre tours de dialogue et les workflows multi-étapes. Une boucle manuelle aurait nécessité de réimplémenter ces mécanismes avec un risque de perte de contexte entre sessions et une gestion complexe des états intermédiaires.

3.3 Architecture globale

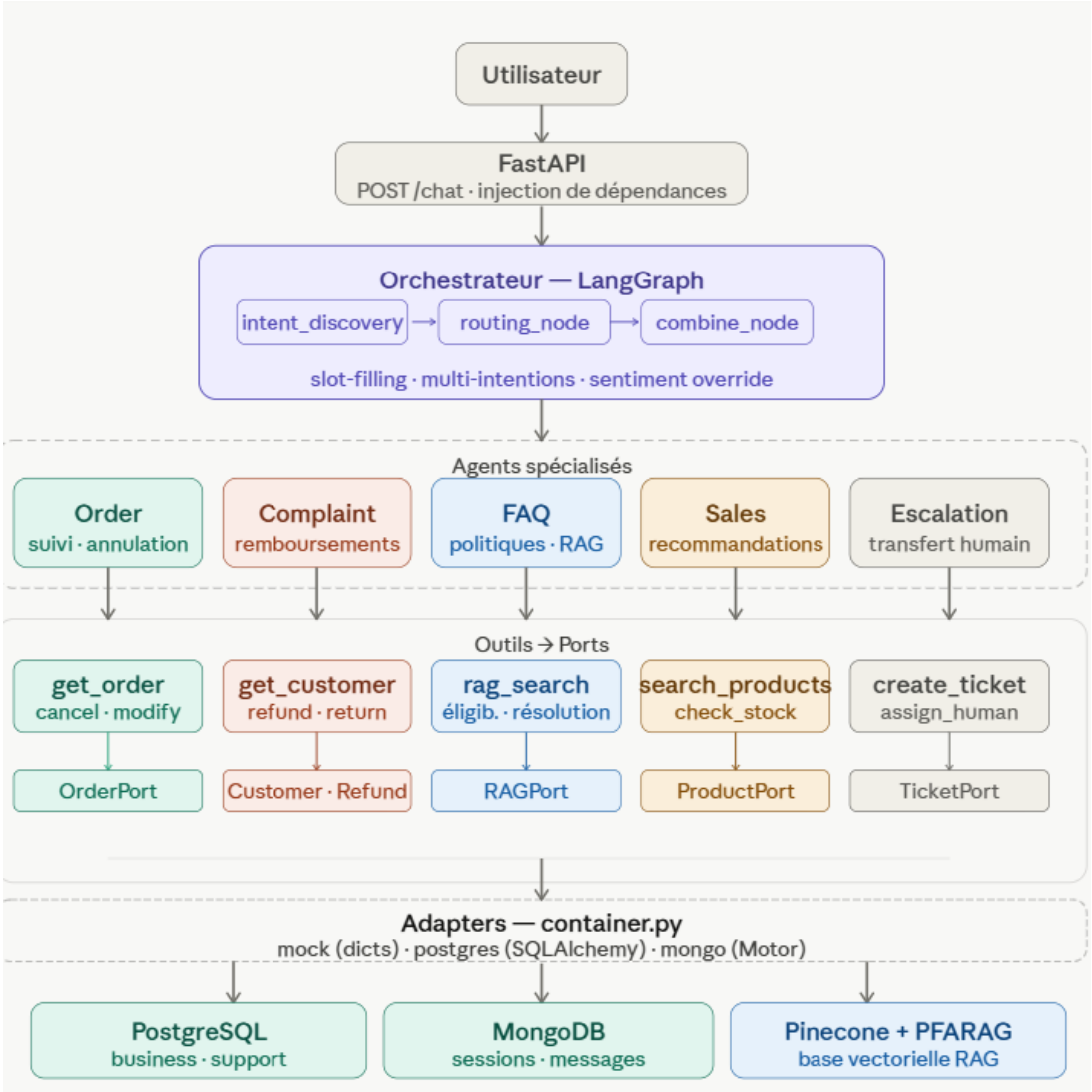


FIGURE 3.2 – Architecture globale du système

La requête client traverse sept couches successives (Figure 3.2), chacune ayant un rôle précis et une frontière claire avec les couches adjacentes.

Utilisateur Le client envoie un message en langage naturel via l'interface chatbot. Le message peut contenir plusieurs intentions simultanées par exemple une demande de remboursement et une question sur une politique de retour dans la même phrase.

API Layer — FastAPI L'endpoint `POST /chat` reçoit la requête HTTP, récupère ou initialise l'état conversationnel LangGraph, et injecte le message dans le graphe. C'est également ici que le conteneur de dépendances (`container.py`) a déjà injecté les adapters dans tous les modules d'outils au démarrage de l'application.

Orchestrateur — LangGraph Le cœur du système. Trois nœuds s'enchaînent selon un graphe conditionnel :

- `intent_discovery` : le LLM analyse le message, identifie les intentions avec un score de confiance, et pose des questions de clarification si nécessaire
- `routing_node` : construit un `AgentContext` par intention et appelle les agents spécialisés en séquence
- `combine_node` : fusionne les réponses des agents en un message cohérent et met à jour le résumé conversationnel

Un mécanisme de *slot-filling* collecte les entités manquantes (identifiant de commande, raison du retour, etc.) avant tout routage. Un *sentiment override* court-circuite le routage normal si la colère dépasse le seuil critique.

Agents spécialisés Chaque agent hérite de `BaseAgent` et exécute une boucle LLM + outils sur son domaine exclusif. Le LLM décide quels outils appeler et dans quel ordre ; `execute_tools()` valide et exécute les appels, puis renvoie les résultats au LLM pour la prochaine itération. La boucle est plafonnée à 8 itérations. Chaque agent retourne un `AgentResult` structuré : `status`, `action`, `response`, `summary`, `tools_called`.

Outils → Ports Les outils sont des fonctions async appelées par les agents. Ils ne connaissent pas la source des données ; ils appellent uniquement les méthodes définies par les interfaces abstraites (*Ports*). Par exemple, `get_order()` appelle `OrderPort.find_order()` sans savoir si derrière se trouve un dictionnaire en mémoire ou une requête PostgreSQL. Un invariant critique garantit que les outils d'action (écriture) ne peuvent jamais s'exécuter dans le même tour qu'un outil de lecture.

Adapters — container.py Au démarrage, `container.py` lit la variable `DB_BACKEND` et instancie l'ensemble des adapters concrets. En mode `mock`, les adapters lisent des dictionnaires en mémoire — aucune infrastructure requise. En mode `postgres`, ils utilisent SQLAlchemy async et Motor (MongoDB). Changer de backend revient à modifier une seule ligne dans le fichier `.env`.

Couche de données Trois systèmes de stockage coexistent, chacun choisi pour ses points forts :

- **PostgreSQL (Supabase)** : données métier (commandes, clients, produits, remboursements) et données support (tickets, événements) dans deux schémas distincts ; les jointures croisées alimentent le dashboard d'administration
- **MongoDB** : sessions et messages de conversation, modèle document naturellement adapté aux données append-only
- **Pinecone + PFARAG** : base vectorielle pour le RAG ; les chunks de documents sont indexés une fois et interrogés dynamiquement par les agents à chaque décision de politique

3.4 Indépendance vis-à-vis des sources de données

Dès le début du projet, nous avons fait le choix de concevoir le système de façon à ce que les agents et les outils ne sachent jamais d'où viennent les données qu'ils manipulent. Ce choix est motivé par le besoin d'avoir un système portable qui peut être déployé pour une autre plateforme e-commerce sans modifier la logique des agents.

La solution adoptée inspirée par **l'architecture hexagonale** [7] repose sur un principe simple : chaque accès aux données passe par une **interface abstraite** (un *Port*), et c'est uniquement au démarrage de l'application qu'une implémentation concrète (un *Adapter*) est branchée derrière cette interface.

> **Principe fondamental**
>
> Les outils appellent des interfaces. Les interfaces ne connaissent pas les bases de données. Les bases de données ne connaissent pas les agents. Toute la logique agentique est donc indépendante de toute infrastructure de stockage.

En pratique, au démarrage de l'application, le fichier `config/container.py` lit la variable d'environnement `DB_BACKEND` et branche les implémentations concrètes :

Listing 3.1 – Branchement des sources de données au démarrage

```
def build() -> dict:
    if os.getenv("DB_BACKEND") == "postgres":
        return {
            "order":    PostgresOrderAdapter(repos.order),
            "customer": PostgresCustomerAdapter(repos.customer),
            "rag":      PFARAGAdapter(),
        }
    return {
        "order":    MockOrderAdapter(),
        "customer": MockCustomerAdapter(),
        "rag":      MockRAGAdapter(),
    }
```

La valeur `mock` branche des dictionnaires en mémoire chargés depuis les données de test. La valeur `postgres` branche les repositories SQLAlchemy connectés à Supabase. Les agents, les outils, les prompts, les décisions — rien de tout cela ne change entre les deux modes.

Conséquence directe Passer du mode développement au mode production revient à changer une ligne dans le fichier `.env`. Adapter le système à une nouvelle plateforme e-commerce revient à implémenter les interfaces abstraites pour la nouvelle source de données, sans toucher à un seul agent.

Les détails des interfaces (ports), des implémentations (adapters) et de la couche de persistance seront présentés dans le chapitre dédié aux données.

Conclusion

Ce chapitre a posé les fondations techniques du système : une architecture multi-agent orchestrée par LangGraph, dont la logique agentique est totalement découplée de toute infrastructure de stockage grâce au mécanisme d'injection de dépendances. Les décisions de conception présentées — spécialisation par agent, RAG dynamique, indépendance des données — ne sont pas des choix théoriques mais des réponses directes aux contraintes rencontrées pendant le développement. Le chapitre suivant présente l'environnement documentaire et transactionnel construit pour évaluer ce système dans des conditions réalistes.

Chapitre 4

Données et environnement documentaire

Introduction

Un système multi-agent raisonne sur deux types de données fondamentalement différents : les **données transactionnelles** (commandes, clients, remboursements) que les agents consultent et modifient via leurs outils, et les **données documentaires** (politiques, procédures, règles métier) que les agents interrogent via le pipeline RAG pour prendre des décisions cohérentes.

Ce chapitre présente la conception de ces deux environnements et explique comment ils ont été construits pour couvrir l'ensemble des scénarios métier de la plateforme StyleSphere.

4.1 Données documentaires

Les agents du système ne connaissent aucune règle métier en dehors de ce qu'ils trouvent dans leur base de connaissances. La qualité des décisions dépend donc directement de la qualité des documents disponibles. Cette section présente comment ces documents ont été construits, enrichis de scénarios réalistes, puis indexés pour être interrogés par les agents.

4.1.1 Le problème des données documentaires

Les politiques internes d'entreprises e-commerce réelles sont rarement accessibles publiquement. Les datasets disponibles sont souvent trop simplifiés pour évaluer correctement un système de raisonnement documentaire. Les environnements réels présentent des difficultés que les données synthétiques doivent reproduire :

Ce que les vrais documents contiennent

- Politiques contradictoires entre départements
- Documents obsolètes jamais supprimés
- FAQs incomplètes ou mal mises à jour
- Règles avec exceptions non documentées
- Styles rédactionnels hétérogènes

Ce que notre approche synthétique apporte

- Conflits contrôlés et reproductibles
- Couverture complète des cas limites
- Niveaux de maturité documentaire variés
- Scénarios calibrés pour chaque agent
- Évaluation rigoureuse possible

4.1.2 Pipeline de génération synthétique

Nous avons développé une pipeline en quatre étapes (Figure 4.1) pour générer des corpus documentaires réalistes. L'idée centrale est de **séparer la structure du contenu** : définir d'abord quels conflits existent, puis générer les documents qui les incarnent. Une génération directe produirait des documents trop homogènes — le LLM convergerait naturellement vers des politiques cohérentes.

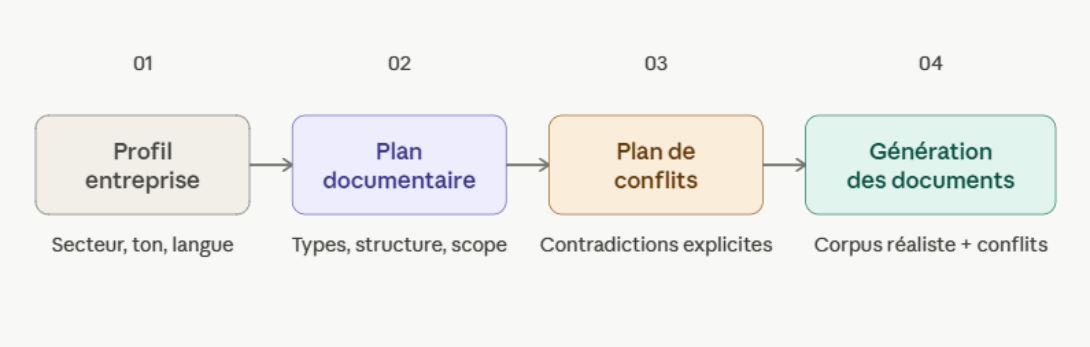


FIGURE 4.1 – Pipeline de génération synthétique de corpus documentaires

TABLE 4.1 – Étapes de la pipeline de génération synthétique

01	Profil d'entreprise	Secteur, taille, régions d'opération, niveau de maturité documentaire, style de communication. Ce profil conditionne toutes les étapes suivantes.
02	Plan documentaire	Inventaire des documents à créer : types (FAQ, SOP, procédures de retour, notes internes), états (actif, obsolète, brouillon), thématiques couvertes. Aucun contenu généré à ce stade.
03	Plan de conflits	Liste des incohérences à introduire entre documents : quelle règle contredit quelle autre, dans quel document, avec quelle date d'effet. C'est l'étape clé pour reproduire la complexité documentaire réelle.
04	Génération des documents	Le LLM génère chaque document individuellement en respectant le profil, le plan et les conflits définis. La cohérence globale est garantie par la structuration préalable.

4.1.3 Le document source — Manuel StyleSphere

Le document de référence utilisé pour évaluer notre système a été produit en appliquant la pipeline de génération synthétique décrite ci-dessus. En partant d'un profil d'entreprise e-commerce , en planifiant l'architecture documentaire, puis en introduisant des conflits contrôlés entre sections, nous avons obtenu le **StyleSphere Operations Manual v4.0** — un manuel interne de 10 sections couvrant l'ensemble des procédures opérationnelles de la plateforme. Ce document est la seule source de vérité pour les décisions de politique des agents : ils ne connaissent aucune règle métier en dehors de ce qu'ils y trouvent.

TABLE 4.2 – Sections du manuel StyleSphere et agents concernés

1	Gestion des comptes et FAQ	FAQ Agent
2	Retours, remboursements, remplacements	Complaint Agent
3	Anomalies de livraison	Order + Complaint Agent
4	Qualité produit, défauts, authenticité	Complaint Agent
5	Compensation par tier client	Complaint + Escalation Agent
6	Litiges vendeur/acheteur	Escalation Agent
7	Paiements et fraude	Complaint Agent
8	Directives de communication	Tous les agents
9	Opérations internationales EU/UK	Complaint Agent
10	Matrice d'escalade interne	Escalation Agent

Trois cas limites de ce document ont été particulièrement utiles pour évaluer la robustesse des agents :

- **Section 9.2 — Directive EU/UK** : le droit de rétractation de 14 jours prime sur la règle Final Sale pour les clients européens — l'agent doit détecter le pays du client avant d'appliquer la règle générale
- **Section 5.2 — Matrice de compensation** : les montants varient selon le tier client (Bronze à Platinum) et la sévérité du problème (mineur, modéré, sévère) — l'agent doit croiser deux dimensions avant de décider
- **Section 3.1 — Non-livraison** : trois chemins de résolution différents selon la valeur de commande (<\$100, \$100–\$500, >\$500) — l'agent doit calculer avant de répondre

4.1.4 Infrastructure RAG — de la politique à la décision

Architecture du service

Le pipeline RAG [3] est déployé comme un **service HTTP indépendant** (PFARAG), séparé du système multi-agent. Cette isolation permet de mettre à jour les documents sans redéployer les agents (Figure 4.2), d'authentifier chaque accès par token Bearer, et de scaler le service indépendamment.

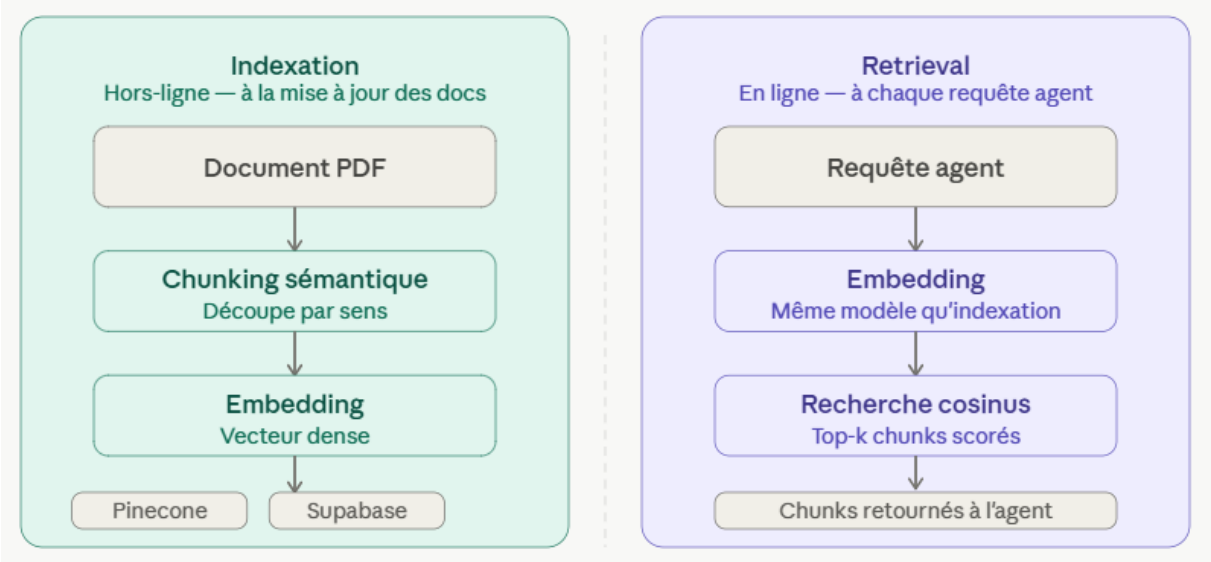


FIGURE 4.2 – Pipeline d'indexation et de retrieval du service PFARAG

Chunking sémantique

Le document est découpé en segments alignés sur les unités de politique — une règle, une procédure, un cas — et non sur des tailles fixes de caractères. Chaque chunk correspond à une information que l'agent peut utiliser seule pour prendre une décision partielle.

TABLE 4.3 – Pipeline de prétraitement et d'indexation

1	Chargement PDF	Texte brut extrait section par section
2	Nettoyage	En-têtes, numéros de page et artefacts supprimés
3	Chunking sémantique	Segments alignés sur les unités de politique
4	Génération des embeddings	Vecteur de dimension fixe par chunk
5	Indexation Pinecone	Chunks interrogeables par similarité cosinus
6	Persistance Supabase	Documents bruts conservés pour traçabilité

Double recherche RAG — contribution clé

Un agent traitant une demande de remboursement effectue deux recherches successives dans un ordre précis.

Recherche d'éligibilité

Vérifier si la demande est *recevable* : fenêtre de 30 jours, état de l'article, catégorie produit.

Recherche de résolution

Déterminer le chemin applicable — *uniquement* si l'éligibilité est confirmée.

Pourquoi cette séparation est critique — Lors des premiers tests avec une recherche unique, l'agent approuvait des remboursements sur des commandes livrées 46 jours auparavant — hors fenêtre — parce qu'il lisait la procédure de résolution *avant* de vérifier l'éligibilité. La double recherche ordonnée a éliminé ce comportement. Un seuil de confiance à `0.6` filtre les chunks non pertinents et force l'agent à reformuler sa requête si nécessaire.

4.2 Données transactionnelles

4.2.1 Architecture de stockage

La couche de données repose sur deux systèmes de stockage complémentaires, chacun choisi pour ses propriétés intrinsèques.

PostgreSQL (Supabase) [8]

Données relationnelles organisées en deux schémas distincts dans la même instance, permettant des jointures croisées indispensables pour le dashboard d'administration.

Schéma `business` :
customers · products · orders · order_items
shipments · shipment_events · refunds · returns

Schéma `support` :
tickets · ticket_events · human_agents
compensations · security_events

MongoDB [9]

Données conversationnelles append-only, sans jointures. Le modèle document est naturellement adapté : chaque session est un document autonome, chaque message embarque son contexte complet.

Collection `sessions` :
session_id · customer_id · statut
résumé sentiment · agent principal

Collection `messages` :
rôle · contenu · agent · tools_called
scores sentiment · timestamp

Pourquoi deux bases différentes —

PostgreSQL est choisi pour les données métier car elles nécessitent des jointures complexes et des contraintes d'intégrité référentielle — un remboursement doit obligatoirement référencer une commande existante. MongoDB est choisi pour les conversations car ce sont des données append-only sans jointures : on écrit un message, on relit la séquence. Les deux paradigmes sont fondamentalement incompatibles — les forcer dans le même système serait un mauvais compromis.

4.2.2 Données de test — conception orientée scénarios

Les données de test ne sont pas aléatoires (voir Figure 4.3). Chaque enregistrement a été conçu pour déclencher un chemin de décision spécifique dans les agents. L'objectif était de couvrir l'intégralité des cas prévus par les politiques StyleSphere, y compris les cas limites les plus complexes.

TABLE 4.4 – Distribution des données de test et scénarios couverts

Clients	30	4 tiers loyalty, 3 comptes `is_flagged`, 6 clients EU/UK (directive droit de rétractation)
Commandes	80	Tous statuts, commandes dans/hors fenêtre 30 jours, valeurs <50 TND, 100–500 TND, >500 TND
Produits	25	Articles hygiènes sensitifs, Final Sale, ruptures de stock, bijoux percés
Remboursements	20	Auto-approuvés, approbation manuelle, BNPL (14 jours), refus, double remboursement bloqué
Retours	15	Article défectueux, mauvais article, changement d'avis, retour EU/UK sur Final Sale
Tickets	60	Toutes priorités, toutes raisons d'escalade, assignés et non assignés

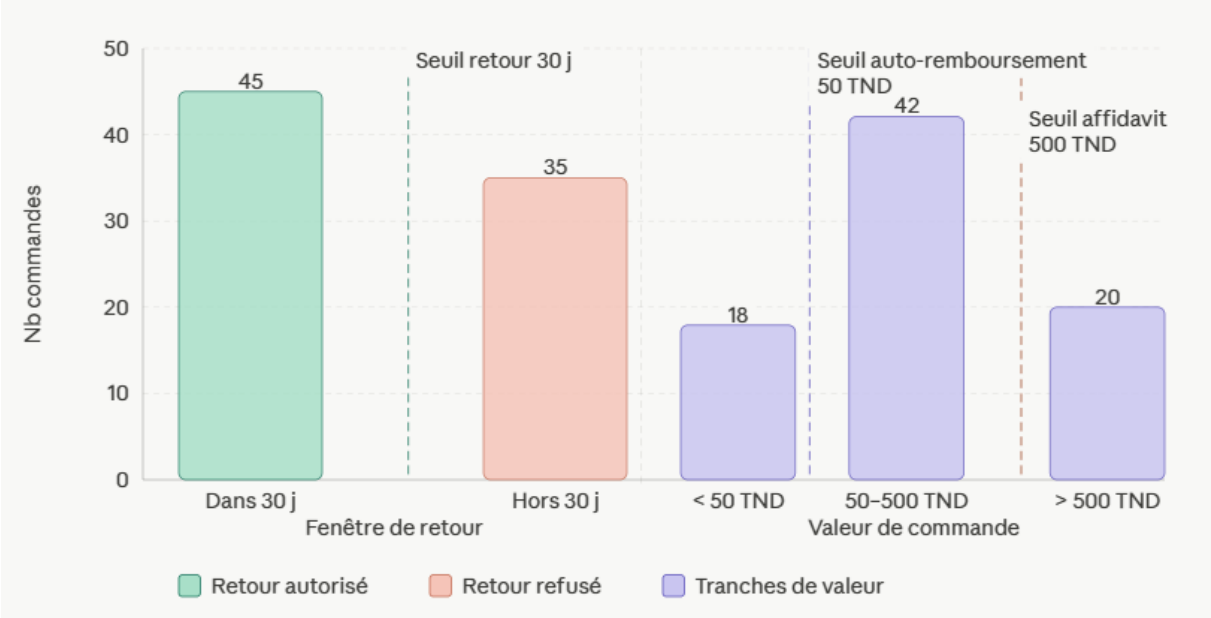


FIGURE 4.3 – Distribution des données de test par rapport aux seuils de politique StyleSphere

Conclusion

Les deux environnements de données présentés dans ce chapitre forment ensemble le substrat sur lequel les agents raisonnent. Les données transactionnelles couvrent l'intégralité des chemins de décision définis dans les politiques StyleSphere. Le pipeline RAG garantit que les agents accèdent aux bonnes informations documentaires au bon moment,

avec une séparation éligibilité/résolution qui a directement amélioré la fiabilité des décisions. Le chapitre suivant présente l'orchestrateur — le composant qui coordonne ces agents et ces données en un système conversationnel cohérent.

Chapitre 5

L'orchestrateur

Introduction

Les chapitres précédents ont posé les fondations du système : l'architecture, les données transactionnelles et documentaires. Ce chapitre marque un tournant — nous passons de la conception à la réalisation. Nous présentons ici le premier composant que nous avons implémenté et qui gouverne l'ensemble du système : **l'orchestrateur**.

L'orchestrateur est le cerveau du système. Aucun agent spécialisé ne reçoit jamais un message directement — tout transite par l'orchestrateur, qui comprend, planifie, délègue, et synthétise. C'est lui qui transforme un message en langage naturel en un plan d'exécution structuré, coordonne les agents spécialisés, et retourne une réponse cohérente au client.

Orchestrateur — rôle et périmètre

- **Entrée :** message brut du client en langage naturel
- **Responsabilité :** comprendre, planifier, déléguer, synthétiser
- **Technologie :** LangGraph `StateGraph` avec checkpointing [6]
- **Sortie :** `ChatResponse` — réponse finale au client
- **Invariant :** aucun agent n'est appelé sans que tous ses champs requis soient collectés

5.1 Architecture interne — les trois nœuds

L'orchestrateur est implémenté comme un graphe d'états LangGraph (`StateGraph`) opérant sur un état `OrchestratorState`. Trois nœuds s'enchaînent selon des arêtes conditionnelles (Figure 5.1) :

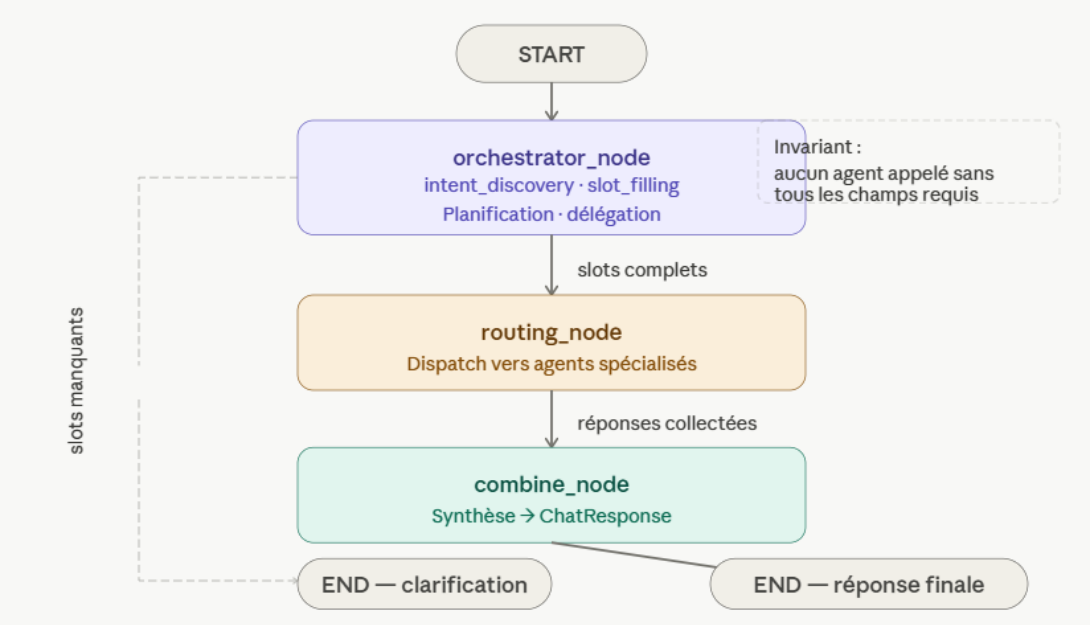


FIGURE 5.1 – Graphe d'exécution LangGraph de l'orchestrateur

TABLE 5.1 – Les trois nœuds de l'orchestrateur

Nœud	Phase	Responsabilité
`orchestrator_node`	intent_discove slot_filling	Analyse le message, détecte les intentions, collecte les champs manquants, pose les questions de clarification
`routing_node`	routing	Construit les `AgentContext`, appelle les agents en séquence, gère les escalades et les `pending_customer`
`combine_node`	done	Fusionne les réponses des agents, met à jour le résumé conversationnel, retourne le `ChatResponse` final

Les arêtes conditionnelles dirigent le flux entre les nœuds :

— `orchestrator_node` → `routing_node` si `phase == "routing"` — toutes les intentions sont confirmées et tous les champs sont collectés

— `orchestrator_node` → `END` si clarification nécessaire — une réponse intermédiaire est envoyée au client

— `routing_node` → `combine_node` si `phase == "done"` — tous les agents ont répondu

— `routing_node` → `END` si `status == pending_customer` — un agent attend une information du client

5.2 Phase 1 — Détection des intentions

Lorsqu'un message arrive, `orchestrator_node` appelle le LLM pour détecter les intentions présentes dans la demande. Chaque intention reçoit un **score de confiance**.

Confiance haute (≥ 0.85)

L'intention est confirmée et ajoutée au plan d'exécution. Le système passe directement au slot-filling pour collecter les champs requis.

Confiance basse (< 0.85)

L'orchestrateur pose une question de clarification au client. Maximum **10 questions** avant escalade automatique vers un agent humain.

Consideration multi-intent

Une même requête peut contenir plusieurs intentions simultanées. L'orchestrateur les détecte toutes et construit un plan d'exécution séquentiel pour chacune.

« Mon colis est arrivé endommagé et je cherche une nouvelle robe pour un mariage. »

Intention 1

`complaint.damaged_item`

score : 0.92 → ComplaintAgent

Intention 2

`sales.recommendation`

score : 0.87 → SalesAgent

5.3 Phase 2 — Slot-filling

Avant d'appeler un agent, l'orchestrateur vérifie que tous les champs requis pour cette intention sont disponibles. Chaque champ manquant déclenche **une question**, posée un champ à la fois.

TABLE 5.2 – Champs requis par intention

`order.track`	`order_id`
`order.cancel`	`order_id`, `reason`
`complaint.refund`	`order_id`, `reason`, `damage_description`
`complaint.return`	`order_id`, `reason`, `condition`
`sales.recommendation`	`category` (optionnel)
`order.modify`	`order_id`, `modification_type`, `modification_value`

Invariant de routage — Aucun agent n'est jamais appelé avec des champs manquants. Le routage ne se déclenche que lorsque tous les `IntentSlots` ont `missing_fields = []`.

5.4 Phase 3 — Routage et exécution

Une fois toutes les intentions confirmées et tous les champs collectés, `routing_node` prend le relais et orchestre l'exécution des agents spécialisés.

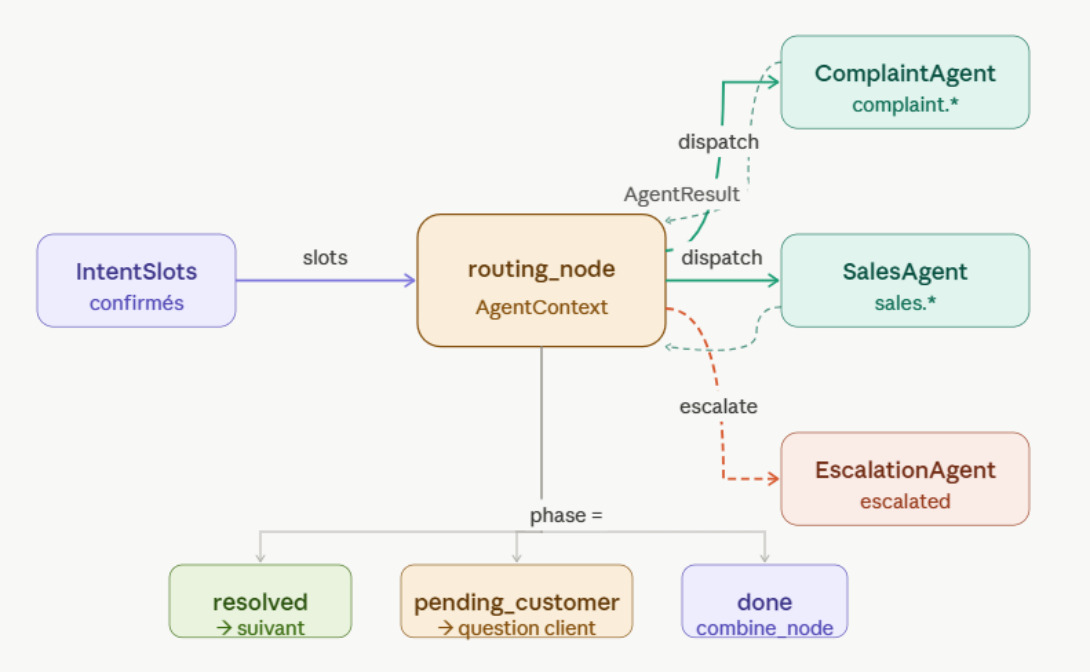


FIGURE 5.2 – Flux d'exécution du routing$_n$*ode*

Pour chaque `IntentSlot` confirmé, le routage suit cinq étapes (Figure 5.2) :

TABLE 5.3 – Étapes du routing_node

#	Étape	Détail
1	Construction du contexte	Profil client, entités extraites, sentiment courant, 6 derniers messages → `AgentContext`
2	Sélection de l'agent	`INTENT_TO_AGENT` map résout l'agent responsable selon le préfixe d'intention
3	Appel de l'agent	L'agent exécute sa boucle LLM + outils et retourne un `AgentResult` structuré
4	Traitement du résultat	`resolved` → suivant · `escalated` → EscalationAgent chaîné · `pending_customer` → arrêt et question au client
5	Fin du cycle	Tous les agents exécutés → `phase = done` → `combine_node`

TABLE 5.4 – Table de routage INTENT_TO_AGENT

Préfixe d'intention	Agent assigné	Domaine
`order.*`	OrderAgent	Commandes
`complaint.*`	ComplaintAgent	Remboursements
`faq.*`	FAQAgent	Politiques
`sales.*`	SalesAgent	Catalogue
`escalation.*` *statut escalated*	**EscalationAgent** *EscalationAgent (chaîné)*	Transfert humain Auto-déclenchement

5.4.1 Sentiment override

En parallèle du routage normal, le sentiment est évalué à chaque tour. Deux conditions court-circuitent le plan d'exécution et déclenchent une escalade directe :

Colère ≥ 0.8

Le client est très agité. Une réponse automatique risque d'aggraver la situation. L'EscalationAgent est appelé directement avec la raison `angry_customer`, sans passer par les agents spécialisés.

Client VIP non résolu

Un client Platinum ou Gold dont le problème n'a pas été résolu au tour précédent déclenche une escalade prioritaire immédiate, indépendamment du score de sentiment.

5.4.2 Le cycle pending_customer

Certains agents ont besoin d'une information supplémentaire du client avant de finaliser leur décision — une photo du produit endommagé, la confirmation de l'état de l'article. Ils retournent alors `status = pending_customer`.

TABLE 5.5 – Déroulement du cycle pending_customer

Tour	Phase	Action
N	routing_node	L'agent retourne `pending_customer` avec une question pour le client
N	END	La question est transmise au client — le cycle s'arrête ici
N+1	orchestrator_node	Détecte `phase == agent_pending_customer`, extrait la réponse et la réinjecte dans le slot
N+1	routing_node	Reprend depuis l'agent en attente avec le contexte complété

5.4.3 Combine node

Une fois tous les agents exécutés, `combine_node` produit la réponse finale.

— **Fusion des réponses** — les résultats de tous les agents sont combinés en un seul message cohérent, sans répétition ni contradiction

— **Mise à jour du résumé conversationnel** — un bloc de texte d'au maximum **10 000 caractères** sert de mémoire interne entre les cycles ; il est enrichi à chaque cycle sans alourdir le contexte du LLM

— **Retour du ChatResponse** — `response`, `agent`, `action`, `session_id` sont retournés au client

5.5 Contrats orchestrateur-agents

La communication entre l'orchestrateur et les agents repose sur deux modèles Pydantic validés à chaque échange. Ces contrats garantissent qu'un agent reçoit toujours un contexte complet et retourne toujours une réponse structurée exploitable.

AgentContext — ce que l'orchestrateur envoie

Champ	Type	Contenu
`customer`	CustomerContext	Tier loyalty, historique achats, flags fraude, VIP
`entities`	EntityContext	order_id, montant, type réclamation, description
`sentiment`	SentimentContext	Scores anger/urgency/frustration + `tone_to_use`
`history`	list[Message]	6 derniers messages de la conversation
`intent`	str	Intention principale détectée par l'orchestrateur
`previous_agent`	str | None	Agent ayant traité le tour précédent

AgentResult — ce que l'agent retourne

Champ	Description
`status`	`resolved` · `escalated` · `handoff` · `pending_customer`
`action`	Action effectuée : `refund_processed`, `return_created`, `order_cancelled`...
`response`	Message destiné au client
`summary`	Résumé concis pour le ticket d'audit
`tools_called`	Liste ordonnée des outils appelés dans ce tour
`handoff_to`	Agent cible si `status = handoff`

Pourquoi deux modèles distincts — *L'`AgentContext` est pré-chargé par l'orchestrateur avant l'appel — l'agent ne fait jamais de requête redondante pour récupérer le profil client ou l'historique. L'`AgentResult` force chaque agent à se positionner explicitement sur son statut — l'orchestrateur sait toujours quoi faire ensuite sans interpréter du texte libre.*

5.6 Tests et évaluation

5.6.1 Scénario 1 — Requête multi-intentions sur deux commandes distinctes

« Je veux suivre une commande et en annuler une autre. »

TABLE 5.6 – Déroulement — deux commandes distinctes

1	intent_discovery	Deux intentions détectées : `order.track` (0.91) et `order.cancel` (0.88)
1	slot_filling	Ambiguïté détectée — les deux intentions peuvent concerner la même commande ou deux distinctes. Clarification posée au client.
2	slot_filling	Client confirme deux commandes différentes. `order_id` collecté pour `order.track` : `TN-0031`
2	slot_filling	`order_id` collecté pour `order.cancel` : `TN-0044` — `reason` manquant, question posée
3	slot_filling	Client fournit la raison — tous les slots complétés
3	routing	OrderAgent appelé deux fois en séquence, une fois par `IntentSlot`
3	combine	Deux réponses fusionnées — suivi de TN-0031 et confirmation d'annulation de TN-0044

Résultat

L'orchestrateur a correctement distingué deux commandes distinctes dans une même requête, collecté les champs séparément pour chaque intention, et routé vers le même agent deux fois avec des contextes différents.

5.6.2 Scénario 2 — Client ne se souvient pas de son identifiant de commande

« Je veux suivre ma commande, mais j'ai oublié le numéro. »

TABLE 5.7 – Déroulement — order_id inconnu du client

1	intent_discovery	Intention `order.track` détectée (0.89). Client signale explicitement qu'il ne connaît pas son `order_id`.
1	slot_filling	L'orchestrateur détecte le signal d'oubli et substitue `order.find` à la demande de `order_id` — aucune question répétitive posée
1	routing	OrderAgent appelé avec l'intention `order.find` et le `customer_id` uniquement
1	routing	`help_user_find_his_order()` retourne la liste des commandes du client avec statuts et articles
2	agent_pending	Client identifie sa commande parmi la liste — `order_id = TN-0027` réinjecté dans le slot
2	routing	`order.track` reprend avec le contexte complet

Résultat

L'orchestrateur n'a pas bloqué sur un champ manquant — il a adapté le plan d'exécution en substituant `order.find` pour aider le client à identifier sa commande, sans demander répétitivement un identifiant que le client ne possède pas.

5.6.3 Scénario 3 — Acquittement simple

« Merci, c'est parfait. »

TABLE 5.8 – Déroulement — message d'acquittement

1	intent_discovery	Intention `meta.acknowledgement` détectée — aucune intention métier identifiée
1	intent_discovery	Court-circuit activé : l'orchestrateur génère directement une réponse de clôture sans construire de plan d'exécution
1	END	Aucun agent spécialisé appelé — réponse retournée directement depuis l'orchestrateur

Résultat

Un message d'acquittement ne déclenche aucun appel LLM vers les agents spécialisés. L'orchestrateur détecte le pattern `meta.acknowledgement` et répond directement, évitant une latence inutile et un appel d'agent sans valeur ajoutée.

Conclusion

L'orchestrateur est le composant qui donne au système sa cohérence conversationnelle. Il permet de traiter des messages complexes et multi-intentions, de guider le client vers les informations manquantes sans le brusquer, et de coordonner plusieurs agents spécialisés en un échange fluide. Sa principale limite reste la latence cumulée : chaque question de slot-filling ajoute un tour de dialogue, et les intentions multiples multiplient les appels LLM. Le chapitre suivant présente les agents spécialisés que l'orchestrateur coordonne.

Chapitre 6

Les agents spécialisés

Introduction

L'orchestrateur coordonne, mais ce sont les agents spécialisés qui décident et agissent. Ce chapitre présente les cinq agents du système, en commençant par la classe abstraite dont ils héritent tous, puis en détaillant le rôle, la conception et les tests de chacun.

6.1 BaseAgent — la boucle partagée

Tous les agents héritent de `BaseAgent`. La boucle agentique est écrite une seule fois (Figure 6.1). Chaque agent spécialisé déclare uniquement trois choses :

- **SYSTEM_PROMPT** — les règles métier, le comportement attendu et les contraintes spécifiques au domaine
- **AGENT_TOOLS** — les schémas OpenAI Function des outils auxquels le LLM a accès dans ce domaine
- **MAX_TOOL_CALLS** — le plafond d'itérations LLM pour cet agent (défaut : 8)

6.1.1 L'invariant action-blocking

Règle de sécurité fondamentale Les outils d'action — `cancel_order`, `process_refund`, `create_return`, `create_ticket`, `assign_human` — ne peuvent jamais s'exécuter dans le même tour qu'un outil de lecture. `execute_tools()` bloque toute tentative et retourne une erreur explicite au LLM, qui doit alors séparer les appels en deux tours distincts.

TABLE 6.1 – Classification des outils — lecture vs action

Type	Outils	Règle d'exécution
Lecture	`get_order` · `get_customer` · `check_stock` · `rag_search`	Parallèle autorisé
Action	`cancel_order` · `process_refund` · `create_return` · `create_ticket` · `assign_human`	Seul, après les lectures

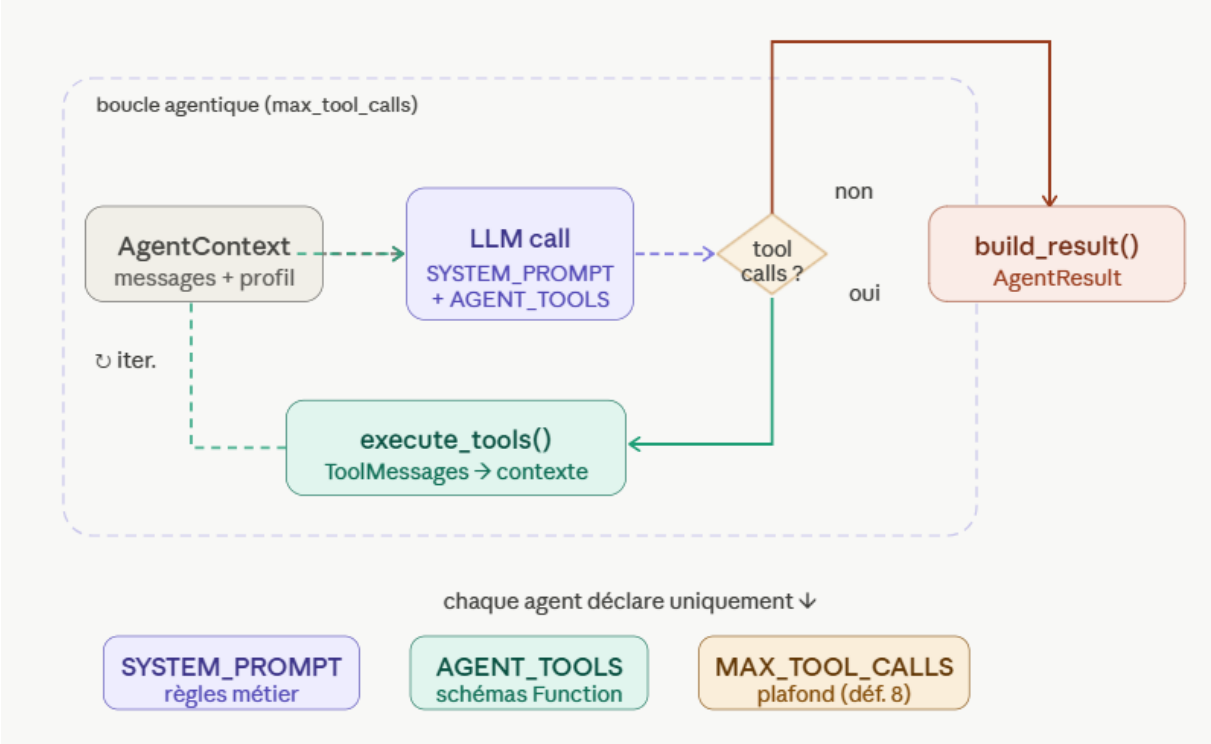


FIGURE 6.1 – Boucle agentique BaseAgent

6.2 Complaint Agent

Complaint Agent — périmètre

- **Domaine :** remboursements, retours, articles endommagés, fraude
- **Autorité :** seul agent autorisé à déclencher un remboursement
- **Seuil auto :** configurable — au-delà, escalade obligatoire
- **Outils :** `get_order` · `rag_search` · `process_refund` · `create_return` · `escalate_ticket` · `create_ticket`

6.2.1 Conception

Le Complaint Agent est l'agent le plus complexe du système. L'agent apprend à vérifier avant d'agir, à distinguer une inférence d'une vérification réelle, et à détecter les signaux de fraude sans les mentionner au client.

Double recherche RAG

Éligibilité — fenêtre 30 jours, état article, catégorie produits. Si non éligible : arrêt immédiat.

Résolution — procédure applicable, montants, délais, preuves requises. Lancée uniquement après .

Verify-before-act

L'agent ne peut pas inférer l'état d'un article — il doit le demander explicitement au client (`pending_customer`).

Une commande livrée hier n'est pas automatiquement dans la fenêtre — l'agent vérifie la date via `get_order` avant toute décision.

TABLE 6.2 – Matrice de décision du Complaint Agent

Situation	Décision	Action
Éligible · montant $\leq$ 50 TND	Auto-approuvé	`process_refund`
Éligible · montant $>$ 50 TND	Approbation requise	`escalate_ticket`
Hors fenêtre 30 jours	Non éligible	Refus poli, `create_ticket`
Client EU/UK · Final Sale	Exception directive 9.2	Retour accepté
Signaux fraude détectés	Escalade silencieuse	`escalate_ticket` (Trust & Safety)
État article non vérifié	En attente	`pending_customer`

6.2.2 Tests

TABLE 6.3 – Scénarios de test — Complaint Agent

Scénario	Comportement attendu	Résultat
Retour changement d'avis, état non précisé	Agent demande confirmation état article avant d'agir	✓ `pending_customer`
Commande livrée il y a 46 jours	Vérification date via `get_order`, refus immédiat	✓ refus propre
Article non conforme, pas de photo	Agent demande preuve photo avant de traiter	✓ `pending_customer`
Client avec 4 remboursements en 30 jours	Escalade silencieuse, ton neutre, aucune mention fraude	✓ escalade discrète

6.3 Order Agent

Order Agent — périmètre

- **Domaine :** suivi, annulation, modification de commandes
- **Volume :** intention la plus fréquente du système
- **Outils :** `get_order` · `track_shipment` · `cancel_order` · `modify_order` · `find_products_skus` · `help_user_find_his_order` · `create_ticket`

6.3.1 Conception

L'agent dispatch ses outils selon l'intention reçue. Trois règles de sécurité structurent son comportement :

- **Vérification de propriété** — `get_order()` vérifie que `order.customer_id == context.customer_id` avant toute action. Une commande appartenant à un autre client retourne une alerte sécurité.
- **Statuts annulables** — l'annulation n'est autorisée que pour les statuts `pending`, `confirmed`, `processing`. Une commande `shipped` ou `delivered` est redirigée vers le Complaint Agent.
- **Confirmation obligatoire** — avant toute annulation, l'agent retourne `pending_customer` pour confirmer l'intention du client.

6.3.2 Tests

TABLE 6.4 – Scénarios de test — Order Agent

Scénario	Comportement attendu	Résultat
Annulation commande `in_transit`	Refus — statut non annulable, redirection Complaint	✓
Modification adresse commande `processing`	Confirmation demandée puis `modify_order` exécuté	✓

6.4 FAQ Agent

FAQ Agent — périmètre

- **Domaine :** questions sur les politiques, procédures, délais, produits
- **Contrainte :** aucun outil d'action — lecture seule
- **Outils :** `rag_search` · `create_ticket`

6.4.1 Conception

Le FAQ Agent est le plus contraint du système — il ne peut que répondre, jamais agir. Trois mécanismes définissent son comportement :

- **Seuil de confiance 0.6** — en dessous, l'agent refuse de répondre et reformule sa requête RAG avant un second essai
- **Retry automatique** — si le second essai dépasse le seuil, la réponse est fournie ; sinon, l'agent indique qu'il ne peut pas répondre et crée un ticket
- **Détection d'action déguisée** — si la question implique une action (*"puis-je retourner ma commande ?"* = demande de retour), l'agent produit un `handoff` vers le Complaint Agent

6.4.2 Tests

TABLE 6.5 – Scénarios de test — FAQ Agent

Scénario	Comportement attendu	Résultat
Question politique de retour	Réponse fondée sur chunk RAG avec source citée	✓
« Je veux retourner ma commande »	Handoff vers Complaint Agent détecté	✓
Score RAG 0.48 au premier essai	Reformulation et second essai — score 0.71	✓ retry

6.5 Sales Agent

Sales Agent — périmètre

- **Domaine :** découverte produit, recommandations personnalisées
- **Contrainte :** jamais de recommandation sur article en rupture
- **Outils :** `search_products` · `get_product_details` · `check_stock` · `get_customer_purchase_history` · `create_ticket`

6.5.1 Conception

Le Sales Agent impose un ordre d'exécution strict en trois phases :

1. **Recherche + historique (parallèle)** — `search_products` et `get_customer_purchase_histo` s'exécutent simultanément
2. **Vérification stock** — `check_stock` sur chaque candidat ; les articles en rupture sont automatiquement exclus
3. **Ticket d'audit** — `create_ticket` seul, en dernier

L'action-blocking du BaseAgent empêche le LLM de recommander un SKU inventé — `check_stock` retourne une erreur si le SKU n'existe pas, et la recommandation est bloquée avant d'être émise.

6.5.2 Tests

TABLE 6.6 – Scénarios de test — Sales Agent

Scénario	Comportement attendu	Résultat
LLM propose SKU inexistant	`check_stock` retourne erreur — SKU bloqué	✓
Article recommandé en rupture	Exclu, alternative en stock proposée	✓
Client avec 6 achats précédents	Recommandation alignée sur l'historique d'achat	✓

6.6 Escalation Agent

Escalation Agent — périmètre

— **Domaine :** transfert structuré vers un opérateur humain
— **Contrainte :** séquence d'outils stricte et non négociable
— **Outils :** `summarize_context` · `set_priority` · `create_ticket` · `assign_human`

6.6.1 Conception

L'agent impose une séquence en trois étapes — toute déviation est bloquée par l'action-blocking :

Étape 1 (parallèle)

`summarize_context` + `set_priority` s'exécutent simultanément.

Étape 2 (seul)

`create_ticket` avec le résumé et la priorité de l'étape 1.

Étape 3 (seul)

`assign_human` avec le `ticket_id` créé à l'étape 2.

6.6.2 Tests

TABLE 6.7 – Scénarios de test — Escalation Agent

Scénario	Comportement attendu	Résultat
Remboursement > 50 TND	Résumé complet, priorité `high`, agent assigné	✓
Client Platinum non résolu	Priorité `urgent`, agent VIP sélectionné	✓

Conclusion

Les cinq agents spécialisés forment un ensemble cohérent où chaque domaine est clairement délimité. Le BaseAgent garantit que tous partagent les mêmes invariants de sécurité, pendant que chaque agent apporte sa logique propre via son prompt et ses outils. Le chapitre suivant présente les perspectives de déploiement en production et les adaptations nécessaires pour passer d'un système de démonstration à un système opérationnel.

Chapitre 7

Réalisation et perspectives

Introduction

Ce chapitre présente l'ensemble de ce qui a été concrètement implémenté au cours du projet — le backend multi-agent, la couche de données, les interfaces utilisateur — puis projette le système dans un contexte de déploiement réel et de commercialisation.

7.1 Stack technologique

TABLE 7.1 – Technologies utilisées dans le projet

Couche	Technologie	Rôle
Orchestration	LangGraph	Graphe d'états, checkpointing, workflows multi-étapes
LLM	Replicate API	Inférence Claude/Llama selon le modèle configuré
RAG	PFARAG + Pinecone	Service HTTP indépendant, recherche vectorielle
API	FastAPI (async)	Endpoint `POST /chat`, injection de dépendances
Base relationnelle	PostgreSQL — Supabase	Données métier et support, deux schémas

7.2 Le système multi-agent

Le cœur du projet est un backend Python entièrement asynchrone exposant un endpoint conversationnel. Le système traite chaque message en passant par l'orchestrateur

LangGraph (Figure 7.1), qui coordonne les six agents spécialisés selon les intentions détectées.

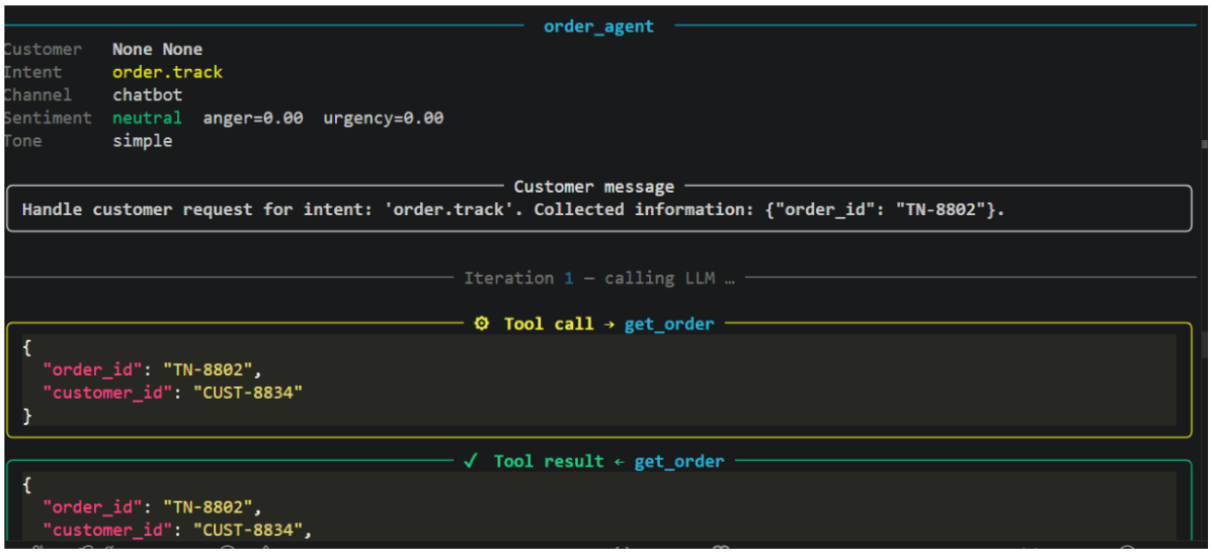


FIGURE 7.1 – Exemple d'exécution de l'agent Order

7.3 La couche de données pluggable

La base de données est entièrement interchangeable à l'exécution. Deux backends sont disponibles :

Mode mock — DB_BACKEND=mock

Dictionnaires Python en mémoire, chargés depuis le jeu de données de test. Aucune infrastructure requise. Démarre en moins d'une seconde. Utilisé pour le développement et l'évaluation des agents.

Mode production — DB_BACKEND=postgres

SQLAlchemy async + Motor connectés à Supabase et MongoDB. Même interface, mêmes agents, mêmes outils. La migration complète (schema + seed data) s'exécute en une commande.

alembic upgrade head && python db/seed.py

7.4 Les interfaces de démonstration

7.4.1 Dashboard administrateur

Un dashboard de monitoring a été développé pour visualiser le comportement du système en temps réel. Il s'adresse aux superviseurs et aux équipes support qui souhaitent observer l'activité du chatbot sans intervenir dans les conversations.

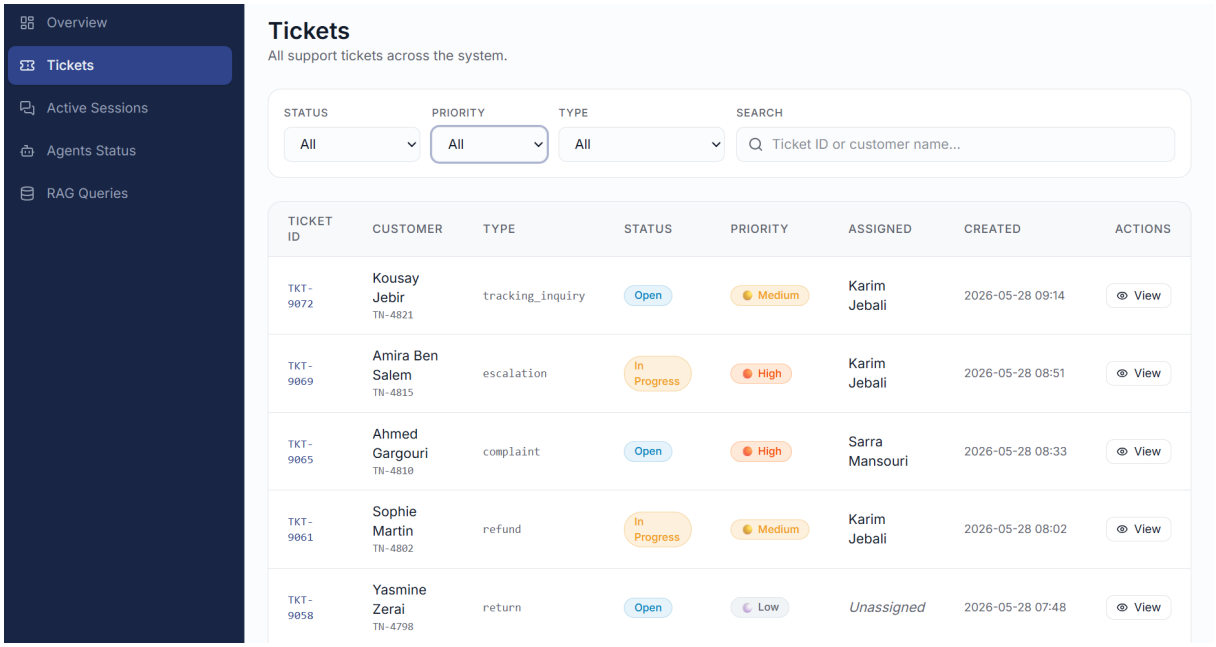


FIGURE 7.2 – Dashboard administrateur — vue Tickets

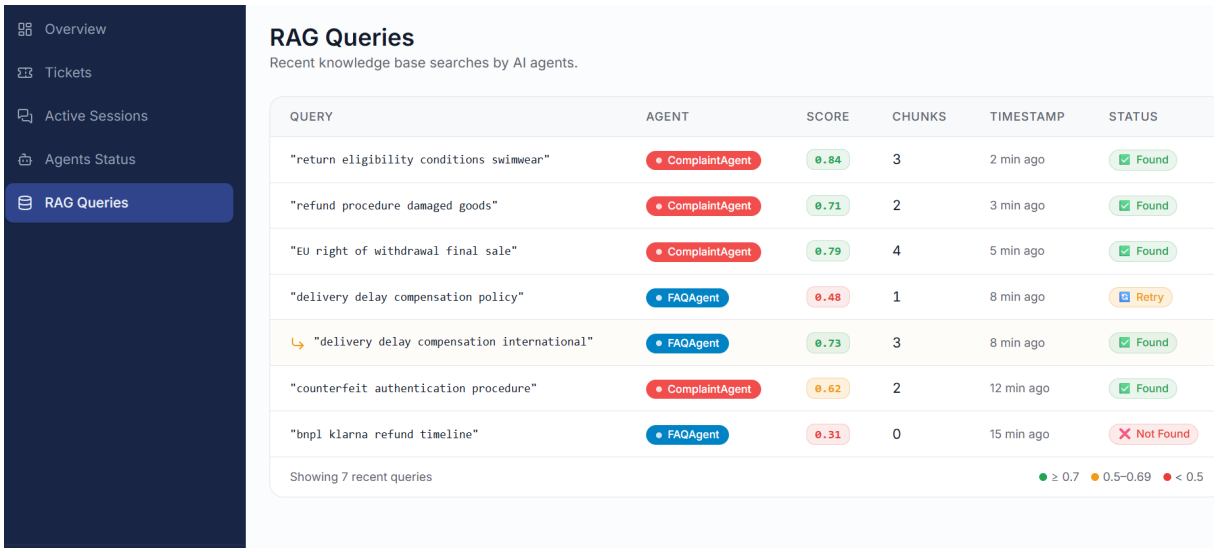


FIGURE 7.3 – Dashboard administrateur — vue RAG

TABLE 7.2 – Pages du dashboard et informations disponibles

Page	Contenu
Tickets	Liste filtrée par statut, priorité, type — badges colorés par sévérité
Sessions actives	Cartes par session avec agent courant, score de sentiment, durée
Statut agents	Disponibilité des agents AI et humains, tailles de queue
Requêtes RAG	Log des recherches avec scores de confiance colorés et statut retry/not found

7.4.2 Widget chatbot client

Un widget conversationnel a été développé pour simuler l'expérience utilisateur finale. Il est connecté directement au backend FastAPI et permet de tester tous les scénarios du rapport en conditions réelles — multi-intentions, slot-filling, escalade, acquittement.

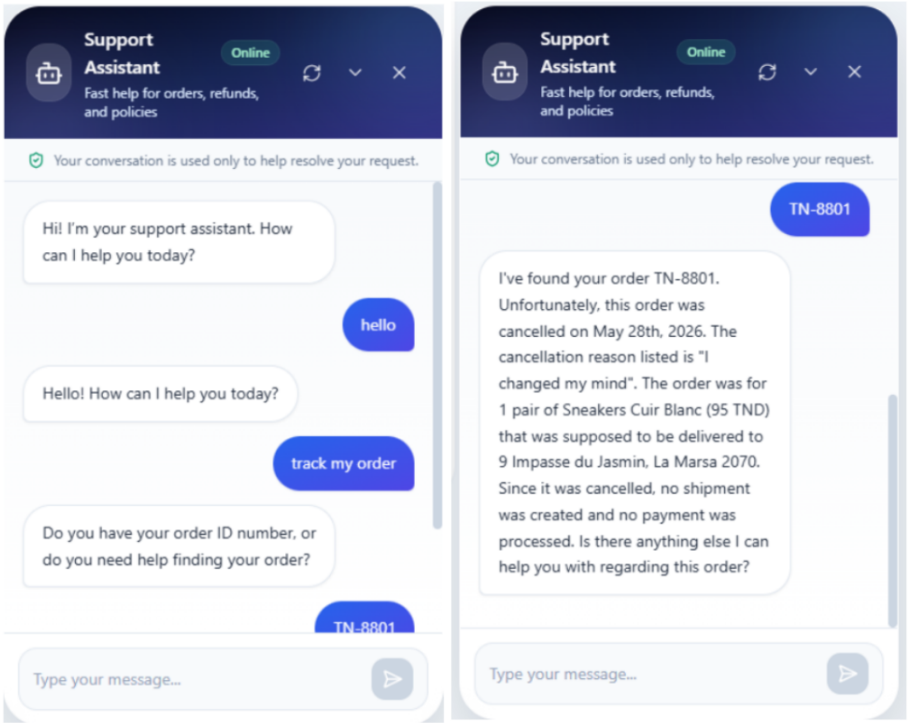


FIGURE 7.4 – Widget chatbot — interface client de démonstration

7.5 Projection produit

7.5.1 Un système conçu pour être vendu

L'architecture Ports & Adapters n'est pas un choix purement technique — c'est un choix commercial. Chaque composant du système a été conçu pour être réutilisable par n'importe quelle plateforme e-commerce :

1. **Implémenter les 9 ports** avec les repositories de la nouvelle base de données — les agents et outils ne changent pas
2. **Indexer les documents de politique** de la nouvelle entreprise dans le service RAG — les agents s'adaptent automatiquement
3. **Configurer les variables d'environnement** — modèle LLM, seuils, backend de données

> **Valeur produit**
>
> *Un retailer e-commerce peut déployer ce système avec ses propres données et politiques en changeant uniquement les adapters et les documents RAG. La logique de raisonnement des agents, l'orchestrateur, le pipeline de slot-filling et les invariants de sécurité sont réutilisables sans modification.*

7.5.2 Cas d'usage réels envisagés

TABLE 7.3 – Cas d'usage potentiels en production

Secteur	Cas d'usage principal	Agents concernés
E-commerce mode	Retours, remboursements, suivi	Complaint + Order
Marketplace	Litiges vendeur/acheteur	Escalation + FAQ
Retail alimentaire	Livraison, qualité, fraîcheur	Complaint + Order
Électronique	SAV, garantie, défauts	Complaint + FAQ
Voyage	Annulations, remboursements	Order + Escalation

7.5.3 Roadmap vers un déploiement réel

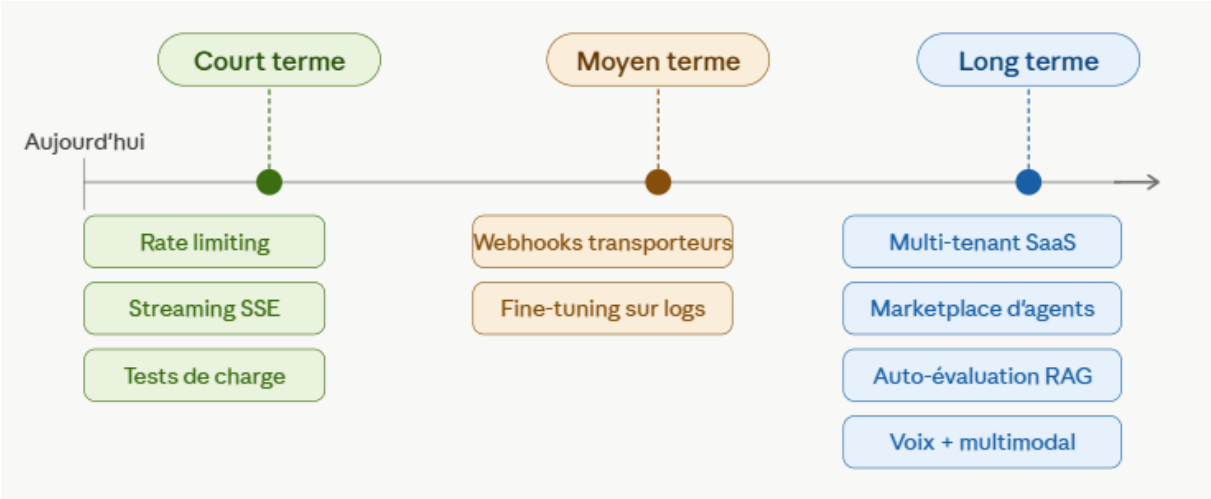


FIGURE 7.5 – Roadmap produit — trois horizons

Conclusion

Ce chapitre démontre que le projet dépasse le cadre académique — l'architecture implémentée, les interfaces réalisées et la portabilité du système constituent une base solide pour un produit commercial. Le chapitre suivant tire les conclusions générales du travail réalisé.

Conclusion

Ce projet démontre qu'il est possible de concevoir un système de support client automatisé capable de traiter des demandes complexes et multi-intentions dans un environnement e-commerce réel. L'architecture multi-agents orchestrée par LangGraph, combinée à un pipeline RAG dynamique et une couche de données entièrement pluggable, répond aux trois contraintes fondamentales du problème : respecter les politiques métier, maintenir la cohérence conversationnelle, et savoir quand passer la main à un humain.

Les cinq contributions techniques du projet — double recherche RAG ordonnée, verify-before-act, action-blocking invariant, Ports & Adapters, et multi-intent slot-filling — ne sont pas des choix théoriques. Ce sont des réponses directes à des comportements incorrects observés lors du développement, corrigés par conception plutôt que par correction de bugs.

Mot final

Une architecture multi-agents bien conçue — responsabilités claires, contrats formalisés, logique totalement séparée de l'infrastructure — peut automatiser un support client complexe avec un niveau de fiabilité et de traçabilité difficile à atteindre avec un agent unique. Ce projet en est la démonstration concrète.

Bibliographie

[1] Tom B. Brown et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901.

[2] Anthropic. *Claude : A Next-Generation AI Assistant*. https://www.anthropic.com/claude. 2024.

[3] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474.

[4] Michael Wooldridge. "An Introduction to MultiAgent Systems". In : (2002).

[5] Lei Wang et al. "A Survey on Large Language Model based Autonomous Agents". In : *Frontiers of Computer Science* 18.6 (2024).

[6] LangChain, Inc. *LangGraph : Building Stateful Multi-Actor Applications with LLMs*. https://langchain-ai.github.io/langgraph/. 2024.

[7] Alistair Cockburn. *Hexagonal Architecture*. https://alistair.cockburn.us/hexagonal-architecture/. 2005.

[8] Supabase, Inc. *Supabase : The Open Source Firebase Alternative*. https://supabase.com/. 2024.

[9] MongoDB, Inc. *MongoDB : The Developer Data Platform*. https://www.mongodb.com/. 2024.